Assignment: Network Discovery and Analysis

Assignment: Network Discovery and Analysis [100 marks]

Target: Metasploitable 2 (deliberately vulnerable Linux VM, isolated lab environment)
Attacker/host machine: Kali/Linux host on the same virtual network (host-only or NAT network, not bridged to a production LAN)

> **Note on this document:** This is a lab report template with the exact commands, expected findings, and the reasoning/analysis answers filled in based on Metasploitable 2's well-documented default configuration. Run each command against your own running VM and paste your terminal output / screenshots into the marked sections before submitting the PDF.

---

0. Lab Setup

1. Download Metasploitable 2 (or 3) and import into VirtualBox/VMware.
2. Set both the attacker VM and Metasploitable to the **same isolated virtual network** (Host-Only Adapter or an internal/NAT network) — never bridge Metasploitable to a real network, it is intentionally full of unpatched vulnerabilities.
3. Boot Metasploitable, log in with `msfadmin` / `msfadmin`, and run `ifconfig` to confirm its IP (or use `netdiscover` from the attacker host, below).

`[ SCREENSHOT: Metasploitable login banner + ifconfig output showing its IP address ]`

---

1. Connectivity Testing with `ping`

```
ping -c 4 <Metasploitable_IP_address>
```

Analysis: - 4/4 packets received, 0% packet loss confirms L3 reachability. - Round-trip time (RTT) will be sub-millisecond to a few ms on a local virtual network (both VMs share a virtual switch), so latency itself isn't diagnostic here — it mainly confirms the ICMP path and that no firewall on either side is dropping ICMP echo. - **If ping fails:** check that both VMs are on the same virtual network (not one NAT + one Host-Only), check `ifconfig` / `ip a` on Metasploitable came up with an IP, and check the attacker host's own interface is on the matching subnet.

`[ SCREENSHOT: ping output ]`

---

2. Path Discovery with `traceroute`

```
traceroute <Metasploitable_IP_address>
```

Analysis: On a single flat virtual network with no intermediate router, expect **one hop** — the Metasploitable IP itself responds directly. This confirms there is no L3 hop (no gateway/router) between the two VMs; both sit on the same broadcast domain. If a virtual router/pfSense VM were placed between them, you'd see that hop listed first.

`[ SCREENSHOT: traceroute output ]`

---

3. Network Discovery and Port Scanning

3.1 `netdiscover` — find Metasploitable's IP

```
sudo netdiscover -r 192.168.x.0/24
```

Replace `192.168.x.0` with your actual lab subnet. Look for the entry whose MAC vendor is Cadmium/VirtualBox/VMware and confirm it against the IP you already saw from `ifconfig` in Metasploitable.

`[ SCREENSHOT: netdiscover results table ]`

3.2 Basic TCP port scan

```
nmap <Metasploitable_IP_address>
```

Metasploitable 2 is famous for exposing a huge number of open ports by design. A typical default (top-1000) scan returns roughly:

Port	Service
21/tcp	ftp
22/tcp	ssh
23/tcp	telnet
25/tcp	smtp
53/tcp	domain
80/tcp	http
111/tcp	rpcbind
139/tcp	netbios-ssn
445/tcp	microsoft-ds (samba)
512,513,514/tcp	rexec/rlogin/rsh
1099/tcp	rmiregistry
1524/tcp	ingreslock (bindshell backdoor)
2049/tcp	nfs
2121/tcp	ftp (proftpd)
3306/tcp	mysql
5432/tcp	postgresql
5900/tcp	vnc
6000/tcp	X11
6667/tcp	irc (UnrealIRCd)
8009/tcp	ajp13
8180/tcp	http (Tomcat)

3.3 Service/version detection

```
nmap -sV <Metasploitable_IP_address>
```

This adds banner-grabbing so each open port is tagged with the actual daemon and version, e.g. `21/tcp open ftp vsftpd 2.3.4`, `80/tcp open http Apache httpd 2.2.8`, `3306/tcp open mysql MySQL 5.0.51a-3ubuntu5`. These specific version strings are what map directly to known CVEs.

`[ SCREENSHOT: -sV output ]`

3.4 NSE vulnerability scan

```
        nmap --script vuln <Metasploitable_IP_address>
```

This runs NSE's vulnerability-detection scripts against every open port. Expect it to flag several known Metasploitable issues, most notably:

- **`ftp-vsftpd-backdoor`** on port 21 — vsftpd 2.3.4 contains a maliciously inserted backdoor: a connection where the username contains `:)` opens a root shell listener on port 6200. This is a textbook example of a supply-chain-style intentional backdoor left in an old build for training purposes.
- **`irc-unrealircd-backdoor`** on port 6667 — UnrealIRCd 3.2.8.1 shipped with a backdoored source tarball allowing arbitrary command execution.
- Possibly Samba/ `smb-vuln-*` findings depending on script set and Samba version installed.

`[ SCREENSHOT: --script vuln output, highlight the CVE/backdoor lines ]`

---

4. Packet Capture and Analysis with `tcpdump` / `tshark`

4.1 Identify your virtual interface

```
        ip a
```

Note the interface name attached to the same virtual network as Metasploitable (e.g. `eth1`, `vboxnet0`, `vmnet1`).

4.2 Capture traffic to/from Metasploitable

```
        sudo tcpdump -i <your_network_interface> host <Metasploitable_IP_address> -w
msf_capture.pcap
```

Leave this running in one terminal while you interact with Metasploitable's services in another (see Challenge below), then `Ctrl+C` to stop.

4.3 Filter the capture for common cleartext services

```
tcpdump -r msf_capture.pcap -n tcp port 80 or tcp port 21 or tcp port 22
```

4.4 Filter with tshark

```
tshark -r msf_capture.pcap -Y "ip.addr == <Metasploitable_IP_address>"
tshark -r msf_capture.pcap -Y "tcp.port == <vulnerable_port>"
```

e.g. `tshark -r msf_capture.pcap -Y "tcp.port == 21"` to isolate the FTP control-channel conversation identified from the nmap scan.

`[ SCREENSHOT: tcpdump and tshark filtered output ]`

4.5 Challenge — interact with a vulnerable service while capturing

Examples:

```
# Browse the web server
curl http://<Metasploitable_IP_address>/

# Attempt an FTP connection
ftp <Metasploitable_IP_address>
```

With the capture running simultaneously, then inspect it:

```
tshark -r msf_capture.pcap -Y "http.request or ftp"
```

For FTP you'll see the full plaintext exchange, including `USER` and `PASS` commands with credentials visible in the clear (e.g. `USER msfadmin`, `PASS msfadmin`) if you authenticate — direct proof of why FTP is considered insecure.

`[ SCREENSHOT: tshark showing plaintext FTP USER/PASS or HTTP GET request ]`

---

5. Basic Network Auditing with `netstat` / `ss` (run *on* Metasploitable)

Log into the Metasploitable VM console directly (or via SSH):

```
netstat -tuln
# or
ss -tuln
```

Expect a long list of listening TCP/UDP sockets matching the nmap results above — `0.0.0.0:21`, `0.0.0.0:22`, `0.0.0.0:23`, `0.0.0.0:80`, `0.0.0.0:3306`, `0.0.0.0:5432`, etc. This is the target's own view of what it's listening on, which should correlate 1:1 with what nmap saw from the outside (any mismatch would suggest a firewall/NAT rule filtering something between the two).

`[ SCREENSHOT: netstat -tuln or ss -tuln output from inside Metasploitable ]`

Written Answers

1. Based on your nmap scan, list three different services running on Metasploitable and the ports they are using.

- FTP — vsftpd 2.3.4 on port 21/tcp
- HTTP — Apache 2.2.8 on port 80/tcp
- MySQL — MySQL 5.0.51a on port 3306/tcp

2. Did the `--script vuln` scan identify any potential vulnerabilities? List one and describe what it suggests.

Yes — `ftp-vsftpd-backdoor` on port 21. It indicates the installed vsftpd 2.3.4 binary contains a known malicious backdoor: sending a username containing the smiley-face string `:)` during FTP login triggers the server to open a root shell on TCP port 6200. It suggests the FTP service should never be exposed as-is, and demonstrates the risk of running unverified/tampered software builds.

3. Difference between a TCP connect scan (`-sT`/default) and a SYN scan (`-sS`)? Why prefer SYN scan?

- `-sT` completes the full TCP three-way handshake (SYN → SYN/ACK → ACK) via the OS socket API for every port, then tears the connection down.

- `-sS` (“half-open” scan) sends a SYN, reads the SYN/ACK response to confirm the port is open, then sends a RST instead of completing the handshake.

`-sS` is faster (no full connection setup/teardown overhead) and stealthier — because the connection is never fully established, it’s less likely to be logged by the target application (though it’s still visible to a stateful firewall/IDS). `-sS` also requires raw-socket privileges (root), whereas `-sT` doesn’t.

4. Did you observe any unencrypted protocols (HTTP/FTP) in the captured traffic? What could be exposed?

Yes — FTP (21) and HTTP (80) are both cleartext. Over FTP, the full username/password pair is visible in `USER` / `PASS` commands, as well as filenames and directory listings transferred. Over HTTP, request URLs, form data (including any login credentials submitted via a plaintext login form), cookies, and full page content are all visible to anyone capturing the traffic — nothing is encrypted, so a passive attacker on the same network segment can trivially read it (this is exactly why HTTPS/TLS and SFTP/FTPS exist).

5. How did you use tshark to filter by a specific port identified as open with nmap?

```
tshark -r msf_capture.pcap -Y "tcp.port == 21"
```

This applies a display filter that keeps only packets where either the source or destination TCP port is 21 (the port nmap reported as open for vsftpd), isolating that single conversation out of the full capture for close inspection.

6. Any unusual/unexpected traffic warranting further investigation?

Traffic to port 1524/tcp (`ingreslock`) is worth flagging — Metasploitable ships this port bound to a root bindshell left over from a previous exploitation (a “backdoor of a backdoor” used in training exercises), so any connection attempt or data on that port is inherently suspicious and would warrant investigation on a real system. Similarly, unexplained outbound connections initiated *from* Metasploitable (rather than responses to your own probes) would be a red flag, since it would suggest something is already running/beaconing on the box.

7. List three services `netstat` / `ss` showed listening. Do they correlate with nmap’s findings?

- `0.0.0.0:21` (vsftpd)
- `0.0.0.0:80` (Apache)
- `0.0.0.0:3306` (MySQL)

Yes — these match the ports nmap reported as open from the attacker host. This correlation is expected on a flat network with no firewall between the two VMs: what the host is genuinely listening on is exactly what an external scan discovers. A mismatch (nmap sees a port that netstat doesn't show, or vice versa) would point to NAT port-forwarding, a firewall rule, or a load balancer sitting between scanner and target.

8. How does using nmap, tcpdump/tshark, and netstat/ss together give a more comprehensive security picture?

Each tool answers a different question, and together they cover the full external-to-internal view of a system's exposure:

- **nmap** answers *"what does this system look like from the outside, and what's potentially vulnerable?"* — attack-surface enumeration and version fingerprinting.
- **tcpdump/tshark** answers *"what is actually being said over the wire?"* — real-time or captured evidence of protocol behavior, cleartext credential exposure, and the true content of a session, which nmap alone can't show since it only probes rather than observes ongoing traffic.
- **netstat/ss** answers *"what does the system itself believe it's running?"* — ground truth from the host's own kernel socket table, useful for confirming (or catching discrepancies with) what the network-facing tools reported, and for spotting processes/ports that might not be reachable externally (e.g. bound only to localhost) but still represent local attack surface.

Used together, they move from *reconnaissance* (nmap) to *verification/evidence* (tcpdump/tshark) to *host-side confirmation* (netstat/ss) — the same triage flow a real security assessment or incident response follows, rather than trusting any single tool's blind spot.

Resources

- nmap cheat sheet
- Introduction to tcpdump
- netstat command guide